

What Does Each Memory Feature Buy? A Measured Audit of an Agent Memory System

Five opt-in features, one harness, and the prices nobody prints: one decisive win, one working feature with documented bugs, and three that do not earn their cost

Kristian Baer

Northtek Labs (FrostByte Digital LLC), Anchorage, Alaska, USA

info@northtek.io · Code: github.com/NORTHTEKDevs/genome

August 2026

Abstract. Memory systems for AI agents accumulate features: graph retrieval, cross-encoder reranking, automatic consolidation, LLM conflict resolution, temporal structure. Vendors describe these features; almost nobody prices them individually. This paper audits all five optional features of GENOME, an open-source agent memory layer I evaluated in earlier work, measuring each feature's benefit against its cost under a single controlled harness. The results split three ways. The bi-temporal belief layer is a decisive win, and this audit closes the largest caveat from the earlier evaluation: its point-in-time advantage survives messy colloquial language, scoring 0.861 against Memo's 0.593 on as-of questions (paired McNemar $p < 0.0001$, $n = 108$) with 96.3% extraction recall from conversational text. Conflict resolution works, removing 15 of 16 superseded facts and repairing retrieval poisoning, but the audit surfaced three concrete defects, including one lost current value. Three features fail to earn their cost: graph retrieval improves aggregate evidence recall by 0.016 while multiplying ingest cost by roughly three orders of magnitude; cross-encoder reranking reliably improves retrieval on every embedder tested but its answer-level effect is small, inconsistent, and never significant; and the shipped auto-consolidation trigger collapses answer accuracy five-fold at its default target, a loss its synthesis operator does not repair. Two methodological findings apply beyond this system: naive substring auditing of memory stores overstated failure rates five-fold until corrected, and answer-accuracy parity between GENOME and Memo replicated in four independent configurations. All harnesses, datasets, and result tables are in the public repository.

1 Introduction

An agent memory system in 2026 is rarely one idea. It is a write path plus a bundle of optional machinery: a knowledge graph over extracted entities, a reranker on top of vector search, a consolidation policy that summarizes or discards old memories, an LLM that adjudicates contradictions, a temporal model of when facts held. Each feature arrives with a plausible story. Very few arrive with a measurement, and I have not found any vendor that publishes what a feature costs next to what it delivers.

This matters because the features are not free. A graph needs entity extraction on every write. A reranker adds a model to every query. Consolidation deletes information on purpose. Conflict resolution puts an LLM call inside the write path that the base system was designed to keep LLM-free. A practitioner choosing a memory stack today has feature lists to compare and almost no prices.

In earlier work [1] I evaluated GENOME, an Apache-2.0 memory layer whose default write path is a single local embedding, and found its answer accuracy at parity with Memo [2], the most visible LLM-ingestion system, at three to four orders of magnitude lower write cost. That paper measured the default path and left the optional features mostly unexamined; it flagged graph retrieval as unvalidated, reported reranking's benefit as embedder-dependent on two data points, and tested consolidation only in a manual equal-budget setting. This paper finishes the job: every optional feature of the system, measured under one harness, with the feature's cost printed next to its benefit, and failures reported with the same prominence as wins. GENOME is my own system, which cuts both ways: I have every incentive to flatter it, and I audit it here the way I would want a stranger to.

The audit's shape is simple. For each feature I ask what it claims to improve, construct a measurement where that claim can fail, run the feature against its own baseline inside the same harness, and report the paired difference and the cost. Three of the five experiments needed no LLM judge at all; their audits are mechanical, which removes one common objection to LLM-evaluated benchmarks.

2 System and related work

GENOME's core, described fully in [1], stores each incoming message with a local sentence-transformer embedding [6] and retrieves by exact cosine search within a per-tenant scope; the default configuration makes no LLM calls and no network calls at write time. The five optional features audited here are: (a) a **bi-temporal belief layer** that extracts dated fact events and answers "what was true at time T" from structure; (b) **LLM conflict resolution**, which classifies each new fact against existing memories as ADD, UPDATE, DELETE, or NONE; (c) **graph retrieval** over auto-extracted entities; (d) **auto-consolidation**, which triggers when a scope exceeds a size threshold and consolidates down to a target, optionally synthesizing summaries of pruned pairs; and (e) a local **cross-encoder reranker** on retrieval candidates.

The comparison baselines are Memo [2], run at its strongest configuration throughout, full conversation context where it fits, and GENOME's own dense default. The evaluation datasets are LoCoMo [3], LongMemEval [4], and TempBelief, a deterministic bi-temporal benchmark introduced in [1] whose generator and gold data are in the repository. MemGPT [5] and the broader retrieval-augmentation literature motivate several of the audited features; the point of this paper is not that anyone's feature is foolish, but that features have prices and the prices are measurable.

3 Method

Every experiment runs the same discipline as [1]: within a comparison, all systems share the same responder model, the same judge with Memo's published judge prompt at temperature 0, the same embedder, and the same retrieval depth, so the feature under test is the only variable. All LLM roles in this audit run through one provider (OpenRouter) on the same model family as the original protocol (Claude Haiku 4.5; the LongMemEval sweep uses a Sonnet responder to keep the answer step from bottlenecking, as in [1]). Every system embeds with GENOME's shipped local default, `all-MiniLM-L6-v2`, except where the embedder itself is the variable being swept. Accuracy significance is paired McNemar on per-question correctness. Where the audit could be mechanical instead of judged, it is: the conflict and consolidation-store measurements check the final state of the memory store directly, and the graph measurement scores retrieval against LoCoMo's annotated evidence turns with no judge involved.

Cost is reported in the unit that matters for the feature: LLM calls and wall-clock for write-path features, CPU time and record amplification for graph ingest, store size for consolidation. Cross-run absolute numbers are not comparable to [1], which used a hosted encoder for its headline tables; within-run paired deltas are the findings here.

4 The feature that pays: bi-temporal belief under natural language

The strongest result in [1] was also its most vulnerable. The belief layer's point-in-time advantage (0.870 versus Memo's 0.676 on as-of questions) was measured on explicitly dated text, and a first attempt at relative dates ("8 months ago") initially collapsed to a tie before an ingest bug was found and fixed, after which a small re-run led but did not reach significance ($p = 0.137$ at $n = 54$). A skeptic's obvious question stood: does the structure survive how people actually talk?

To answer it I added a third rendering register to the TempBelief generator. The same deterministic fact-event log, the same gold answers, but assertions rendered as conversational speech with hedges, asides, and colloquial but arithmetically resolvable time anchors: "Oh, did I tell you? Diego took a job at a startup in June last year." A third of facts are still narrated out of chronological order, and distractors (tentative plans, one-off events) still must not move belief. The register is committed as `tempbelief_nl.json`; the design rule, unchanged from [1], is that anchors must be resolvable from the message date, because the eval measures resolution, not guessing.

Split	GENOME belief	full-context	GENOME dense	Memo
as-of (point-in-time)	0.861	0.213	0.194	0.593
current-value	0.833	0.833	0.889	0.500
history	0.972	0.778	0.861	0.778

Table 1. TempBelief, natural-language register: 6 conversations, 194 questions (108 as-of). Paired McNemar on as-of: belief vs Memo 35 vs 6, $p < 0.0001$; belief vs full-context 71 vs 1 and vs dense 73 vs 1,

both $p \approx 0$.

The edge survives, and decisively. Belief's as-of accuracy barely moves from the explicit-dated register (0.870 to 0.861) while every baseline stays far behind; the paired test against Memo is no longer trending but conclusive. The mechanical extraction audit holds up as well: the belief store captured 104 of 108 gold fact events from colloquial text, 96.3%, matching the published explicit-register recall almost exactly, so the accuracy reflects a correct store rather than a lenient judge. Belief also wins history ($p = 0.0233$ versus full-context, $p = 0.0455$ versus Memo) and, unusually, current-value against Memo in this register ($p = 0.0033$), though the honest note there is that Memo's current-value fell to 0.500 from 0.861 in the explicit register, and whether that drop comes from the register or from running Memo on the local embedder is unresolved; I flag it rather than claim it.

5 The feature that works, with defects: conflict resolution

ConflictBench is a new deterministic scenario set: sixteen entity-attribute slots, each receiving an assertion, a rephrased duplicate, a correction to a new value, and, for half the slots, a subsequent pure negation. Both arms ingest identical turns; the only difference is `resolve_conflicts`. The audit is fully mechanical: it inspects the final store for positive assertions of each value and asks retrieval one current-value question per slot. No judge model touches the result.

Metric	naive insert	resolve conflicts
current value present	8/8	7/8
stale value positively asserted	16/16	1/16
negated fact positively asserted	8/8	1/8
top-1 retrieval correct	4/8	7/8
records in store	101	34
ingest seconds	72	212

Table 2. ConflictBench final-store audit, two independent runs (second shown; the first reproduced within one slot on every metric). Presence metrics recomputed from dumped stores with the positive-assertion audit.

The control arm quantifies the problem the feature exists to solve: without resolution, every superseded and negated fact remains, and half of top-1 lookups return a stale value. With resolution on, the store keeps or explicitly retracts what it should, retrieval poisoning largely disappears, and the store is a third the size, at 2.9 times the ingest time (one resolution call per extracted fact).

The audit also surfaced three concrete defects, each reproducible from the dumped stores in the repository. One correction was stored only as a retraction ("user previously believed Marcus was a teacher") without asserting the corrected value, which is how the feature lost one current value. One

stale and one negated fact survived as positive assertions. And the extraction layer, independent of resolution, padded the store with thirteen near-empty "user has a relationship with [name]" records and one outright hallucination. These are bug reports with line-item evidence, and they are the kind of finding a feature audit exists to produce.

There is a methodological result here worth more than the feature verdict. My first audit counted value substrings, and it reported resolution failing five of eight negations. That number was wrong: a stored negation ("Ravi no longer owns a pickup truck") contains the value string, and a correction sentence contains both old and new values, so substring presence is not assertion. Restricting the audit to positive assertions (value present, competing value absent, no negation or retraction markers) moved the negation-failure count from five to one. Anyone auditing a memory store with string matching should assume their first number is inflated the same way mine was.

6 A feature that does not pay: graph retrieval

[1] declined to score graph mode because the store under test had never built an entity graph, and an earlier full-pipeline graph variant had scored far worse on answers; the burden of proof was explicitly left with graph. This audit paid the toll: two LoCoMo conversations ingested with entity extraction enabled, then every evidence-annotated question scored for gold-evidence recall under dense and graph retrieval, paired, with no judge.

Category	n	dense	graph	Δ
multi-hop	43	0.281	0.297	+0.016
temporal	63	0.698	0.730	+0.032
single-hop	114	0.395	0.404	+0.009
open-domain	11	0.439	0.439	+0.000
all	231	0.459	0.474	+0.016

Table 3. Evidence hit rate, dense vs graph, top-k 30. Graph returned a different result set than dense on 176 of 231 questions, so the mode is genuinely active.

The benefit is real but marginal: no regression anywhere, a gain of 0.016 overall and 0.016 on multi-hop, the category the graph exists for. The cost is not marginal. Ingesting one 419-turn conversation with entity extraction produced 4,230 records, a tenfold amplification, and consumed roughly 2.9 CPU-hours of graph building plus an extraction call per turn; the dense write path ingests the same turns in about ten milliseconds each with no LLM. Three orders of magnitude of ingest cost for a point and a half of recall is not a trade any deployment should make on this evidence, and dense remains the validated default. I note the scope honestly: two conversations, retrieval-only, one embedder. But the burden of proof sat with graph, and this measurement moves it no closer to meeting it.

7 A feature that harms at its default: auto-consolidation

[1] tested consolidation manually at equal token budget and found synthesis and deletion statistically indistinguishable. But the shipped feature is not manual: `auto Consolidate threshold` fires on its own when a scope outgrows a threshold and consolidates down to a target, and nobody had measured what that does to accuracy mid-stream. I ingested one 419-turn LoCoMo conversation under three arms: the trigger off, the trigger on with synthesis (the default operator), and the trigger on with pruning only, then answered the conversation's 152 non-adversarial questions identically in each arm.

Arm	Accuracy	Final store	Ingest s
off	0.454	1,468	595
synthesize	0.092	243	590
prune only	0.086	243	593

Table 4. Auto-consolidation at threshold 300, target 150 (the constructor's default target). McNemar vs control: synthesize 4 vs 59, prune 2 vs 58, both $p < 0.0001$.

The collapse is total and the mechanism is visible in the store trajectory: from roughly turn 100 the trigger fires continuously, pinning the store near 250 records where the control grows to 1,468, and the discarded 83% contained the evidence the questions needed. Synthesis does not rescue it; 0.092 versus 0.086 reproduces the equal-budget wash of [1] under the feature's own trigger. The honest scope is one conversation and one cap ratio, and a threshold sized far above the workload's working set may well behave benignly; that configuration is untested here. But the practical conclusion is blunt: auto-consolidation is a lossy compression knob, not a free win, and at an aggressive cap it silently trades away most of the system's accuracy. Following this measurement I added a warning to the feature's documentation in the repository, which is what a feature audit is for.

8 A feature that improves the wrong metric: reranking, swept across embedders

[1] reported the reranker's answer-level benefit appearing with a strong hosted embedder and vanishing with a weak local one, on two data points. This audit adds two more local embedders, BGE small and large [7], on the LongMemEval head-to-head at $n = 90$ per leg, with both systems sharing each embedder.

Embedder	dense acc	rerank acc	dense hit@10	rerank hit@10	vs Memo
bge-small (n=90)	0.533	0.556	0.834	0.907	parity ($p=0.83$ / 0.48)

bge-large (n=90)	0.533	0.556	0.896	0.916	parity (p=0.82 / 0.83)
MiniLM (n=205) [1]	0.595	0.546	0.827	0.895	parity (p=0.14)
OpenAI te3-small (n=90) [1]	0.689	0.700	0.917	0.943	parity (p=0.19)

Table 5. Reranking across four embedder configurations. Paired p-values on the new legs are dense-vs-Memo / rerank-vs-Memo. The two [1] rows are prior runs included for the full picture; backends and n differ, so compare within rows.

Three things fall out. First, the reranker reliably improves retrieval on every embedder, by two to seven points of hit@10; as a retrieval feature it is real. Second, its answer-level effect is small and inconsistent, ranging from -0.049 to $+0.023$ across the four configurations and never approaching significance; the earlier "embedder-dependent" story was too generous, and the fair summary is that reranking's end-to-end benefit should not be assumed. Third, and unexpectedly, retrieval is not the bottleneck at these levels: bge-large beats bge-small by six points of dense hit@10 with identical answer accuracy. Once hit@10 clears roughly 0.83, the answering step limits accuracy, which bounds what any retrieval improvement can deliver on this benchmark.

The sweep also produced the audit's most replicated result as a byproduct: GENOME-versus-Memo answer parity held in all four configurations, across two backends and four embedders. A fifth check, rerunning the published head-to-head protocol with its own locked harness (`benchmarks/head_to_head.py`) through the single-key OpenRouter path, was still executing when this version was published; its result will be committed to the repository as `benchmarks/head_to_head_locked.txt` and reported in a revision of this record, whatever it says. On the four completed configurations, the parity claim of [1] is the most independently repeated finding either paper contains.

9 What the audit adds up to

Read as one table, the audit has a shape worth stating. Everything that ships on by default in this system survived measurement, and everything that failed is an opt-in flag. That is partly luck and partly a consequence of the design bet in [1]: the system's thesis is that write-time intelligence is overpriced, and the features that failed here are exactly the ones that reintroduce expensive machinery into the path the thesis stripped bare. The one opt-in that decisively pays, the belief layer, is also the one that adds structure rather than model calls at query time, and its price is paid in a scoped extraction step whose output can be mechanically audited.

Two findings travel beyond this system. Audit design dominates apparent failure rates: my own first audit overstated conflict-resolution failures five-fold, and I would now distrust any store-inspection number produced by bare substring matching. And feature pricing is cheap to do: the entire audit,

five features across six experiments, cost about thirty dollars of API spend and a weekend of machine time, which removes the main excuse for shipping features without prices.

10 Limitations

Every experiment is single-run at fixed seeds; the ConflictBench comparison was run twice and reproduced, but the others were not repeated. Sample sizes are moderate ($n = 90$ to 231 per comparison; 108 as-of questions for the headline result), so small deltas in either direction are not distinguishable from noise, and I have treated them accordingly. TempBelief and ConflictBench are benchmarks I built, with the generators and gold data committed for inspection; self-built benchmarks deserve extra suspicion, and the fact-event vocabulary is narrow. The consolidation result tests one cap ratio on one conversation. The graph result is retrieval-only and does not measure answer accuracy. All new runs use one model family through one provider and a single local embedder except where swept. During this work I also found that the TempBelief generator's use of Python's salted string hash makes bit-exact regeneration depend on a fixed `PYTHONHASHSEED`; the committed datasets are canonical and the requirement is now documented, but it is a reminder that reproducibility bugs hide in generators as easily as in systems. The evaluation was designed and executed by one person with automated tooling and has not been independently replicated; the harnesses are public precisely so that it can be.

11 Reproducing this audit

```
git clone https://github.com/NORTHTEKDevs/genome
cd genome
pip install -e .
export OPENROUTER_API_KEY=sk-or-...
python benchmarks/tempbelief_run_or.py --convs 6 --data benchmarks/data/tempbelief_nl.json -
-tag nl
python benchmarks/conflictbench_or.py --slots 16      # mechanical audit; --reaudit is free
python benchmarks/ab_graph_retrieval_or.py --n-convs 2
python benchmarks/consolidation_scale_or.py --conv 0 --threshold 300 --target 150
python benchmarks/lme_qa_or.py --n 90 --embedder BAAI/bge-small-en-v1.5
python benchmarks/head_to_head.py --provider openrouter --n 10 --q 30
```

One key runs everything; the whole audit costs roughly \$30 at current pricing. LoCoMo and LongMemEval are not redistributed (LoCoMo is CC BY-NC 4.0); `benchmarks/data/README.md` documents how to obtain them. The TempBelief datasets are self-built and committed. Result tables as run for this paper are in `benchmarks/AUDIT-RESULTS.md` and the files it links.

12 Availability

GENOME is Apache-2.0 at github.com/NORTHTEKDevs/genome, on PyPI as [genome-memory](https://pypi.org/project/genome-memory/), and in the official Model Context Protocol registry as [io.github.NORTHTEKDevs/genome](https://github.com/NORTHTEKDevs/genome). The companion paper on the core system is [1]; raw run artifacts for any number here are available on request.

References

- [1] K. Baer. Do Agents Need an LLM to Remember? An Evaluation of LLM-Free Memory Ingestion. Zenodo, doi:10.5281/zenodo.21987934, 2026.
- [2] P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav. Memo: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.
- [3] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753, 2024.
- [4] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, D. Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024.
- [5] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [6] N. Reimers, I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084, 2019.
- [7] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff. C-Pack: Packed Resources For General Chinese Embeddings. arXiv:2309.07597, 2023.

Acknowledgment of process: implementation and experiment execution were done with heavy AI assistance working under my direction; the research questions, fairness decisions, and the choice to publish failures alongside wins are mine, and every number regenerates from the public scripts. Independent replication is welcome and encouraged. Contact: info@northtek.io.